ملخص المحاضرة الأولى

شرح Claude Code للمبتدئين من الصفر

من «يعني إيه Agent؟» حتى الـ Subagents والـ Skills — شرح عملي خطوة بخطوة، حتى لو أول مرة تفتح الترمنال.

الوكيل وحلقته · الصلاحيات · التثبيت · إدارة السياق · CLAUDE.md

Subagents · Skills · أوامر /

المحاضِر
غريب الشيخ

المدة
حوالي 57 دقيقة

المصدر
مبني على كورس Claude Code 101 من Anthropic

رابط الفيديو
youtu.be/fjJ0iM-ft-0

فهرس المحتويات

1 ما هو Claude Code؟ وما هو الـ AI Agent؟

2 المفاهيم الأساسية: السياق، الأدوات، الصلاحيات

3 حلقة الوكيل Agent Loop

4 طرق التثبيت والتشغيل + أوامر الترمنال

5 التطبيق العملي وأوضاع الصلاحيات

6 اختيار النموذج وإدارة السياق والجلسات

7 ملف CLAUDE.md والذاكرة

8 الوكلاء الفرعيون Subagents

9 أهم أوامر / في جدول واحد

10 المهارات Skills والـ Plugins

11 الخريطة الذهنية — صفحة كاملة

12 الخريطة الشجرية — صفحة كاملة

13 ملاحظات سريعة للمراجعة — لا تنسَ

14 أسئلة المراجعة — 30 سؤالاً

15 الإجابات النموذجية

16 عناوين للتوسع والبحث

1 ما هو Claude Code؟ وما هو الـ AI Agent؟

◆ **التعريف**

Claude Code أداة برمجة تعمل **بمنطق الوكيل** (**Agentic coding tool**): تقرأ الـ **Codebase** الخاصة بك كاملة وتكتبها، وتتصل بالأدوات التي تستخدمها أصلاً كمطوّر، لتنجز شغلك بشكل أسرع. وتعمل بأكثر من طريقة وعلى أكثر من منصة.

ما هو الـ AI Agent (وكيل الذكاء الاصطناعي)؟

برنامج يستطيع أن **يتفاعل مع البيئة المحيطة به** وينفّذ أفعالاً ليحقق هدفاً معيناً. الفكرة بسيطة: نموذج ذكاء اصطناعي يعمل داخل **حلقة متكررة** ولديه وصول (**Access**) لمجموعة أدوات.

نموذج لغوي LLM + **أدوات Tools** + **ذاكرة Memory** + **تخطيط Planning** = **وكيل Agent**

أي نموذج مثل **GPT** أو **Opus** أو **Sonnet** هو في الأصل «نموذج لغوي» فقط؛ عندما نضع حوله أدوات تزيد قدراته، وذاكرة، وقدرة على التخطيط — **يتحول إلى وكيل**.

★ **الفكرة المحورية**

Claude Code هو **الوكيل** الذي يعمل في الخلفية، ومعه **نموذج لغوي** من نماذج **Anthropic** (**Opus** أو **Fable** أو **Sonnet** أو **Haiku**) هو «العقل». الوكيل يستخدم هذا النموذج لينفّذ المهام بالأدوات المتصل بها.

ماذا يستطيع Claude Code أن يفعل؟

القراءة والفهم

- يقرأ الـ **Codebase** ويفهمها
- يشرح لك **Feature** معيّنة
- يكتشف **Bug** أو مشكلة في الكود

الكتابة والتعديل

- القراءة والكتابة عنده «سيّان»
- يضيف ملفات ويعدّل داخلها ويحذفها
- يعمل على مستوى المشروع كله

التنفيذ في الترمنال

- يشغّل الأوامر مثلك تماماً (**build**، تشغيل المشروع)
- يشغّل الاختبارات (**Tests**)
- ينزّل الـ **Packages** التي يحتاجها

البحث

- يبحث على الإنترنت
- يقرأ التوثيق (**Documentation**)
- يراجع مراجع الـ **API**

الفرق الجوهري: Claude.ai مقابل Claude Code

وجه المقارنة	Claude.ai (الشات)	Claude Code
طبيعته	تتكلم مع النموذج اللغوي مباشرة	وكيل (**Agent**) يستخدم النموذج
الوصول للملفات	لا يوجد — نسخ ولصق	**وصول مباشر** لملفاتك
الترمنال	لا	يشغّل الأوامر ويثبّت ويختبر
النتيجة	يرد عليك بنص	**يأخذ إجراءً** وينفّذ بدلاً منك

2 المفاهيم الأساسية: السياق، الأدوات، الصلاحيات

لكي تستخدم **Claude Code** بفعالية، ضع في ذهنك هذه المفاهيم:

١. نافذة السياق (Context Window)

- مثل أي نموذج لغوي، له نافذة سياق قد تصل إلى **مليون توكن** تقريباً.
- لكنه **لا يحتاج أن يحمّل المشروع كله** داخل السياق ليفهمه؛ بل يجد طرقاً أذكى ليجيب من داخل الكود.
- تحتوي على: محتوى الملفات، مخرجات الأوامر، البرومتس التي كتبتها، وغيرها — وهو يحدد ما يخزّنه وما يرجّعه لمحادثتك.
- عند امتلائها يعمل **Compact** تلقائياً: يقرر ما يلخّصه أو يحذفه ليرجع لحجم مناسب **دون هلوسة أو فقدان للسياق**.

٢. الأدوات (Tools)

- هي **العمود الفقري** لأي وكيل في الدنيا.
- الشات بوت العادي «يرى نصف الكلام»، أما الأدوات فتجعل **Claude Code** يقرر **متى** ينفّذ الكود، و**متى** يقرأ الملفات، و**متى** يبحث على الإنترنت.
- لا يستدعي الأدوات عشوائياً؛ بل يستخدم **الفهم الدلالي** (**Semantic Understanding**) ليحدد: متى يستدعي الأداة، وكيف يستخدمها، وكيف يتعامل مع مخرجاتها (مثل نتائج البحث).

٣. الصلاحيات (Permissions)

- **افتراضياً** يسألك قبل تشغيل أي أمر أو إجراء أي تغيير في ملفاتك — أنت دائماً «ماسك الدفّة».

- تستطيع أن تتحكم في كل التفاصيل، أو تتركه يعمل وحده بالكامل — حسب اختيارك.

تحذير مهم جداً

Claude Code ليس معصوماً من الخطأ: قد **يفهم قصدك خطأً**، أو **يُحدث Bug** من نفسه، أو يعمل **Over-engineering** ويعقّد الحل أكثر من اللازم. الشهرة الكبيرة للأداة **لا تعني** أن نعتمد عليها دون أن نراجع ما تُخرجه. أي أداة في الدنيا تخطئ.

3 حلقة الوكيل Agent Loop

هذه هي الطريقة التي يعمل بها **Claude Code** داخلياً — خطوة بخطوة:

1 تكتب البرومت

من المتصفح أو الترمنال أو إضافة **VS Code**.

2 جمع السياق (Gather Context)

يتواصل مع النموذج اللغوي (افتراضياً نماذج **Anthropic**: **Opus**، **Sonnet**، **Fable**).

3 رد النموذج: نص أو Tool Call

النموذج **لا يعرف استخدام الأدوات بنفسه**؛ فيرجع للوكيل ويقول له: «استخدم الأداة الفلانية».

4 الإجراء (Action)

ينفّذ **Claude Code** الإجراء: يعدّل ملفاً أو يشغّل أمراً.

5 التحقق (Verification)

يتأكد أن النتائج التي طلبتها في البرومت تحققت فعلاً.

6 انتهاء أو إعادة

إن تحقق المطلوب ← **Done** وينتظر خطوتك التالية. وإن لم يتحقق ← يرجع يجمع السياق ويدخل الحلقة مرة أخرى.

★ الحلقة ديناميكية

أثناء دوران الحلقة تستطيع أن **تضيف معلومات أو سياقاً أو توجّهه لاتجاه آخر**. وهو لا ينتظر حتى ينتهي مما كان يفعله ثم ينظر لرسالتك؛ بل يأخذها في الحسبان **فوراً**.

الملخص: Claude Code يجمع أربعة مفاهيم وكيلية

حلقة وكيل مرنة + نافذة سياق + أدوات + صلاحيات قابلة للضبط

4 طرق التثبيت والتشغيل + أوامر الترمنال

١. تطبيق سطح المكتب

- الأسهل لأي مستخدم (**macOS** أو **Windows**)
- تثبيت عادي كأي برنامج
- تذهب لتبويب **Code** (وليس **Home** الذي هو الشات العادي)

٢. الترمنال (الأسهل عند المحاضِر)

- من صفحة **claude.com/product/claude-code** أو ابحث «**Claude Code**»
- أمر لكل نظام: **Windows CMD**، **PowerShell**، **macOS / Linux / WSL**
- انسخ الأمر ← الصقه ← **Enter** ← **Installation complete**
- للتشغيل اكتب: `claude`

٣. إضافة المحرر

- إضافة (**Extension**) لـ **VS Code** أو منتجات **JetBrains**
- تثبيت كأي إضافة + تسجيل دخول
- تظهر في تبويب جانبي (المحاضِر يستخدم **Cursor**)

٤. الويب

- **claude.ai** ← **Code** ← **Get started**
- من الترمنال: `/web-setup`
- يحتاج حساب **GitHub** ليعمل **clone** و**push**

تسجيل الدخول والخروج

- `/login` ← اختر نوع الحساب: اشتراك (**Pro / Max / Teams**) أو **API** ← **Authorize** من المتصفح.
- عند أول تشغيل يسألك: هل تريد أن يعمل داخل المجلد الحالي؟ وقد يعرض تجربة ميزات جديدة.

- `exit` للخروج والرجوع للمجلد الذي كنت فيه.

أوامر الترمنال الأساسية — لا تخف من «الشاشة السوداء»

الأمر	معناه	الاستخدام
`ls`	list	يعرض محتويات المجلد الذي تقف فيه
`cd اسم`	change directory	الدخول إلى مجلد (مثل `cd Desktop`)
`mkdir اسم`	make directory	إنشاء مجلد جديد (أو أنشئه من الواجهة الرسومية)
`claude`	تشغيل	يشغّل **Claude Code** داخل المجلد الحالي
`exit`	خروج	يغلق **Claude Code**

✔ نصيحة

لا تشغّل **Claude Code** على سطح المكتب مباشرة؛ **اعمل دائماً داخل مجلد مشروع محدد**.

5 التطبيق العملي وأوضاع الصلاحيات

أوضاع الصلاحيات (Permission Modes)

الوضع	تعديل الملفات	أوامر الترمنال	متى تستخدمه
Default / Manual	بإذنك	بإذنك	تريد التحكم في كل خطوة
Accept Edits	**تلقائي**	بإذنك (الأشياء الحساسة)	تثق في التعديلات وتراقب الأوامر
Plan Mode	❌ قراءة فقط	❌	يخطط ويرجع بخطة **To-do** قبل التنفيذ
Auto Mode	**تلقائي**	بموافقتك حسب الشرح	بعد الموافقة على خطة تثق بها

◆ **الاختصار الأهم**

`Shift + Tab` يبدّل بين الأوضاع: **Accept edits on** ← **Plan mode** ← **Auto mode on** ← **Manual**.

تخطي الصلاحيات له تكلفة

يمكنك أن تجعله يعمل «على الطيار الآلي» دون الرجوع إليك، لكن كلما أعطيته حرية كاملة زادت **الأخطاء التي يصعب التقاطها** (**Skipping permissions has a cost**).

التجربة الأولى: ملف فارغ

- داخل مجلد **test** فارغ كتب: «**create an empty txt file**».

- طلب الإذن بثلاثة خيارات: **Yes** — أو **Yes, allow all edits during this session** (يعطيه صلاحية التعديل طوال الجلسة) — أو **No**.
- النتيجة: ظهر ملف **empty.txt** في المجلد ← **Claude Code** يعمل بنجاح.

التجربة الثانية: تطبيق To-do list عبر Plan Mode

1 طلب بسيط في Plan Mode

«**simple to-do list**».

2 أسئلة توضيحية

أداة **CLI**؟ **React**؟ أم ملف **HTML** واحد؟ ← **Single HTML file**. أين التخزين؟ ← محلياً في المتصفح (**localStorage**). ثم **Submit answers**.

3 الخطة

ماذا سيضيف (صندوق إدخال، زر حذف، التخزين، مكان حفظ الملف، طريقة التحقق).

4 اختيار طريقة التنفيذ

Yes + Auto mode — أو **Yes, manually approve edits** — أو **Ultraplan** على **Claude Code on the web** لمناقشة الخطة — أو تعديل الخطة بنفسك.

5 التنفيذ

اختار الموافقة اليدوية؛ يعرض **الثواني والتوكنز المستهلكة**، ثم طلب إنشاء **todo.html** وتشغيل أمر لفتحه في المتصفح.

التجربة الثالثة: إضافة Dark Mode

- دخل **Plan Mode** «ليراجع الخطة قبل كتابة الكود».
- سأله: زر تبديل (**Toggle**) يدوي؟ أم حسب النظام؟ أم الاثنين؟ ← زر يدوي.

- الخطة: تعديلات في **todo.html**، متغيرات **CSS**، أيقونة التبديل، كود **JavaScript**، والتحقق.
- نفّذ بـ **Yes and use auto mode**؛ ويعرض التغييرات: **الأحمر = محذوف** و**الأخضر = مضاف**.

6 اختيار النموذج وإدارة السياق والجلسات

النموذج ومستوى الجهد

- اكتب `/` لتظهر قائمة الأوامر وبجانب كل أمر وظيفته.
- `/model`: اختيار النموذج — **Default (recommended)** وكان **Sonnet 5**، أو **Sonnet** / **Fable** / **Opus** / **Haiku**.
- `/effort`: مستوى الجهد = **عمق التفكير**: **low** ← **medium** ← **high** ← **xhigh** ← **max**.

> ✔ **استراتيجية توفير التوكنز**
>
> استخدم **نموذجاً أقوى** (**Opus** / **Fable**) للتخطيط ووضع الخطة، ثم **بدّل لنموذج أخف** (**Sonnet**) لينفّذ الخطة — لأن النماذج الأعلى تستهلك توكنز أكثر.

مراقبة السياق

- `/context`: يعرض النموذج، وحجم النافذة (قرابة **مليون توكن**)، والمستهلك (مثلاً 48 ألف توكن).
- **Auto-compact**: عند الاقتراب من الامتلاء يحجز مساحة (**buffer** ≈ 33 ألف توكن) ويفتح نافذة جديدة.
- للأمان: اعمل **compact** بنفسك **قبل الامتلاء بقليل — أو قبل المنتصف**.

أوامر الجلسات

الأمر	ماذا يفعل
`/compact`	يأخذ **ملخص** المحادثة ويفتح نافذة سياق جديدة به
`/clear`	جلسة جديدة **بسياق فارغ تماماً**؛ المحادثة السابقة لا تُؤخذ في الاعتبار

الأمر	ماذا يفعل
`/new`	يفتح جلسة جديدة أيضاً
`/resume`	الرجوع لمحادثة سابقة من قائمة (مع خانة بحث)
`/rewind`	الرجوع لنقطة محفوظة: **الكود فقط، أو المحادثة فقط، أو الاثنين**

★ لماذا `/rewind`؟

إذا مشى Claude معك في اتجاه خاطئ، لا تقعد تقول له «reverse the changes» أو «رجّعني للحتة الفلانية» فتحرق توكنز أكثر — استخدم `/rewind` واختر النقطة مباشرة.

7 ملف CLAUDE.md والذاكرة

◆ **ما هو CLAUDE.md؟**

من أكثر الميزات فائدة: ملف يعطي **Claude Code ذاكرة دائمة** عن مشروعك. بامتداد **.md** (**Markdown**) ويوضع في **جذر (root) المشروع** — يمكن تسميته **«دستور المشروع»**.

المشكلة التي يحلها

- كل مرة تفتح **Claude Code** على المشروع **يبدأ من الأول**.
- قد **ينسى قواعد** تريده أن يلتزم بها.
- يعيد **استكشاف الكود** ليعرف طريقتك في الكتابة وتفضيلاتك.
- وأحياناً **يفترض افتراضات** من عنده.

ماذا نكتب فيه؟

- طريقة **تشغيل المشروع**.
- **أسلوب الكود** (**Code style**): مثلاً إزاحة بمسافتين (**2 spaces indentation**)، طريقة تسمية المتغيرات.
- أي تفضيلات لا تريد تكرارها كل مرة — «خلاص هتلتزم بيها 100%».

كيف ننشئه ونعدّله؟

1 يدوياً

ملف جديد باسم **CLAUDE.md** في جذر المشروع.

2 **بطلب من Claude**

«**create CLAUDE.md file in the root**» فيكتب فيه معلومات المشروع.

3 **بالأمر `/init` (الأفضل)**

ينشئ الملف ويضع فيه توثيق الكود الحالي. **اجعله أول خطوة في أي مشروع.**

4 **التعديل بـ `/memory`**

دون إغلاق الجلسة.

نوع الذاكرة	أين؟	نطاقها
Project memory	ملف **CLAUDE.md** داخل المشروع	هذا المشروع فقط
User memory	على مستوى الجهاز/المستخدم	كل المشاريع

8 الوكلاء الفرعيون Subagents

◆ **التعريف**

Subagent وكيل فرعي متخصص يعمل **بالتوازي** ولديه **نافذة سياق خاصة به**، ينجز مهمة محددة ثم يرجع لك بالنتيجة.

- الأمر `/agents` ما زال موجوداً، لكن **المعالج (Wizard) الذي كان يُنشئ الوكلاء أُزيل**.
- الطريقة الحالية: **تطلب مباشرة**، مثل: «اعمل لي **subagent** اسمه **code-reviewer** يراجع الكود ويعطيني ملخص».

سير العمل في المثال

إنشاء code-reviewer ← **«review the code»** ←

تقرير: Critical / Warnings / Minor ← **«based on this, fix...»** ←

مراجعة مرة أخرى

كرر المراجعة والإصلاح حتى لا يتبقى إلا ملاحظات بسيطة جداً.

★ **القاعدة**

أي مهمة تتكرر — اعمل لها Subagent. وبمجرد أن تقول الكلمة المتفق عليها أو تستدعيه، يشغّله ويؤدي مهمته.

9 أهم أوامر / في جدول واحد

التحكم في **Claude Code** ليس بالبرومتس فقط؛ هناك **طبقة تحكم بالأوامر** المدمجة (**Built-in commands**) — اكتب `/` لتظهر القائمة.

الأمر	الوظيفة
`/init`	إنشاء **CLAUDE.md** بتوثيق الكود الحالي
`/memory`	تعديل ذاكرة المشروع أو المستخدم
`/model`	تغيير النموذج
`/effort`	تغيير مستوى الجهد (عمق التفكير)
`/context`	إحصائيات نافذة السياق
`/compact` · `/clear` · `/new`	تلخيص / بدء نظيف / جلسة جديدة
`/resume` · `/rewind`	الرجوع لمحادثة / لنقطة محفوظة
`/doctor`	**فحص صحة شامل**: سلامة الإعداد، **Skills** و**MCP servers** غير مستخدمة تأكل السياق
`/statusline`	شريط ثابت أسفل الشاشة: النموذج، المجلد، **Git branch**، نسبة السياق (مثلاً 7% مستخدم)
`/diff`	التغييرات غير الملتزم بها قبل **commit** أو الرفع لـ **GitHub**
`/security-review`	مراجعة أمنية مبدئية سريعة للثغرات
`/cost` أو `/usage`	تكلفة الجلسة والأسبوع ونسبة كل عملية (مثلاً `/doctor` 15%)
`/btw`	سؤال جانبي دون قطع المحادثة

الأمر	الوظيفة
`/plugin` · `/reload-plugins`	متجر الإضافات والمهارات / إعادة التحميل بعد التثبيت
`/login` · `/web-setup`	تسجيل الدخول / ربط الويب بـ **GitHub**
`/agents`	إدارة الوكلاء الفرعيين

> ✔ **ملاحظة ذكية**
>
> إذا أوقفت أمراً في منتصفه (مثل فحص `/doctor`)، يقترح عليك **Claude Code** برومتاً لاستكمال ما توقف.

10 المهارات Skills والـ Plugins

◆ **ما هي الـ Skill؟**

ملف بسيط جداً (غالباً لا يتعدى **~100 سطر**) يعرّف **Claude كيف ينفّذ مهمة معينة بطريقة معينة**. و«ثلاثة أرباع» الأوامر التي استخدمناها (مثل **code review**) هي في الأصل **Skills**.

الميزة الأساسية

لا تُحمَّل في نافذة السياق افتراضياً؛ تبقى «على جنب»، ولا تُستدعى إلا وقت الحاجة — فتبقى نافذة السياق فارغة.

تكوين الـ Skill

الاسم name

اسم المهارة.

الوصف description

أهم جزء: هو المُشغِّل (**Trigger**)؛ كلماته تحدد متى تعمل المهارة تلقائياً.

الخطوات

ماذا يفعل بعد ماذا حتى يحقق المطلوب.

ثلاث طرق لاستدعاء أي Skill

1 ضمنياً (بالمعنى)

«ابنِ لي موقعاً شخصياً» ← يشمّ رائحة التصميم فيستدعي **frontend-design** تلقائياً.

2 بذكر اسمها

«ابنِ لي موقعاً شخصياً باستخدام **frontend-design skill**».

3 إجبارياً بالأمر

`/` + اسم المهارة.

مثال: مهارة explain-code — تشرح الكود بلغة بسيطة تناسب المبتدئين، ووصفها: «**Use when the user asks what a file or function does**» ← فيكفي أن تسأل في وسط الكلام «الدالة دي بتعمل إيه؟».

متجر الإضافات `/plugin`

- **Marketplace** كبير فيه حوالي **267** مهارة رسمية من **Anthropic**، مع وصف بجانب كل واحدة وخانة بحث.
- نطاق التثبيت: **Project scope** (هذا المشروع) أو **Local scope** أو **User scope** (المستخدم على الجهاز كله).
- بعد التثبيت: `/reload-plugins`.
- أمثلة: **frontend-design**، **superpowers**، **code-review**، **context7**، **skill-creator** (لإنشاء مهاراتك)، **code-simplifier**، **github**، **playwright**، **typescript-lsp**، **vercel**، **security-guidance**.
- ويمكنك تنزيل مهارات من الإنترنت بأخذ ملف **.md** أو تشغيل أمر في الترمنال.

★ **التجربة: قبل وبعد frontend-design**

تصميم الـ **To-do list** الأول كان «واضح جداً إنه **AI**»؛ بعد تثبيت المهارة وطلب **Redesign** خرج تصميم احترافي مبهر (يحتاج تظبيطات بسيطة) — **المهارات تفرق جداً**.

الفرق بين الـ Skills وCLAUDE.mdg

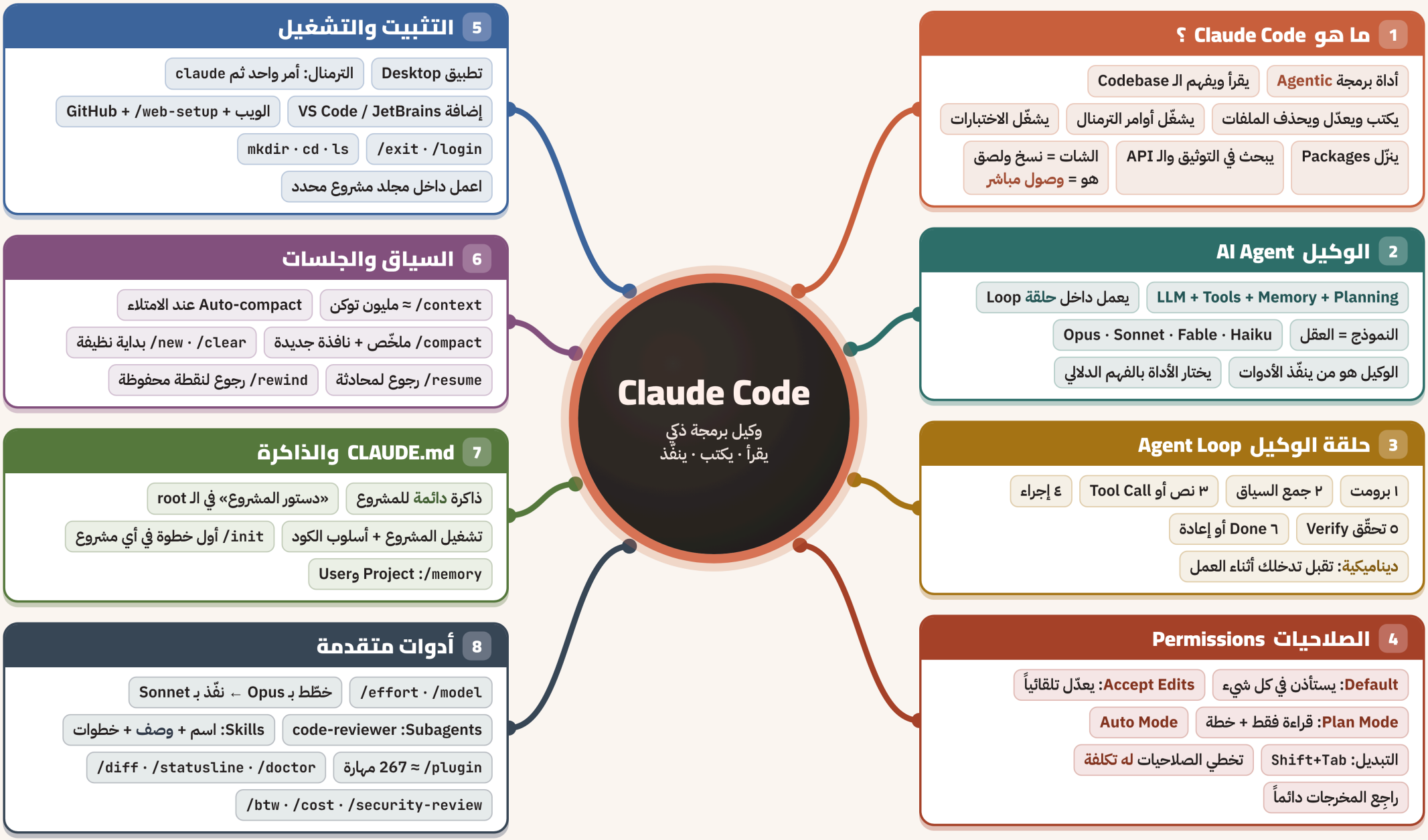
	Skills	CLAUDE.md
الطبيعة	طريقة لتنفيذ **مهمة محددة**	**فلسفة العمل** وقواعد المشروع الأساسية
التحميل	عند الحاجة فقط	**دائماً** داخل السياق

✔ نصيحة ختامية

لبناء منتج حقيقي يُعتمد عليه **لا تكتفِ بـ Sonnet**؛ استخدم نموذجاً مثل **Opus** لتخرج بنتيجة موثوقة دون معاناة.

الخريطة الذهنية — Claude Code للمبتدئين

كل أفكار المحاضرة في صفحة واحدة



الخريطة الشجرية — Claude Code للمبتدئين

تسلسل الموضوعات من العام إلى التفاصيل

Claude Code
كورس المبتدئين
غريب الشيخ

التعريف والفكرة

- **أداة Agentic:** تعمل بمنطق الوكيل
- **القدرات:** قراءة/كتابة كود، تعديل ملفات، أوامر، اختبارات، Packages، بحث
- **مقابل الشات:** وصول مباشر للملفات والترمنال بدل النسخ واللصق
- **تحذير:** يخطئ ويعقّد أحياناً — راجِع دائماً

الوكيل وحلقته

- **الوكيل =** LLM + أدوات + ذاكرة + تخطيط داخل حلقة
- **الحلقة:** برومت ← سياق ← Tool Call ← إجراء ← تحقق ← انتهاء/إعادة
- **الأدوات:** العمود الفقري، تُختار بالفهم الدلالي
- **ديناميكية:** تضيف سياقاً أو توجّهه أثناء العمل

الصلاحيات

- **Default / Manual:** إذن قبل أي تعديل أو أمر
- **Accept Edits:** تعديل الملفات تلقائياً، الأوامر بإذن
- **Plan Mode:** قراءة فقط وخطة To-do
- **Auto Mode:** تنفيذ تلقائي — `Shift+Tab` للتبديل

التثبيت والتشغيل

- **4 طرق:** Desktop · ترمنال · VS Code/JetBrains · ويب
- **الترمنال:** أمر التثبيت من الموقع ثم `claude` و`/login`
- **الويب:** `/web-setup` وربط GitHub للـ clone/push
- **أوامر أساسية:** `ls` `cd` `mkdir` `exit`

التطبيق العملي

- **ملف فارغ:** empty.txt بإذن Yes / allow all edits
- **To-do list:** أسئلة ← خطة ← تنفيذ بموافقة يدوية
- **Dark mode:** خطة في Plan Mode ثم Auto
- **بعد الخطة:** Auto · يدوي · Ultraplan · تعديل

النموذج والسياق

- **`/model`:** Sonnet · Opus · Fable · Haiku
- **`/effort`:** low → medium → high → xhigh → max
- **`/context`:** ≈ مليون توكن + Auto-compact
- **الجلسات:** `/compact` `/clear` `/new` `/resume` `/rewind`

CLAUDE.md

- **المشكلة:** كل جلسة تبدأ من الصفر وتنسى القواعد
- **الحل:** ملف Markdown في جذر المشروع = دستور
- **المحتوى:** طريقة التشغيل، أسلوب الكود، التفضيلات
- **الأوامر:** `/init` للإنشاء، `/memory` للتعديل

الوكلاء والمهارات

- **Subagent:** سياق مستقل بالتوازي (code-reviewer)
- **Skill:** اسم + وصف (المُشغّل) + خطوات، لا تُحمَّل إلا عند الحاجة
- **الاستدعاء:** ضمنياً · بالاسم · بـ /
- **الأوامر:** `/plugin` `/doctor` `/statusline` `/diff` `/cost`

13 ملاحظات سريعة للمراجعة — لا تنسَ

- [ ] **الوكيل** = LLM + أدوات + ذاكرة + تخطيط داخل حلقة. **Claude Code** هو الوكيل، والنموذج هو العقل.

- [ ] النموذج **لا ينفّذ الأدوات**؛ يطلب **Tool Call** والوكيل هو الذي ينفّذ.

- [ ] الحلقة: برومت ← سياق ← نص/**Tool Call** ← إجراء ← **تحقق** ← انتهاء أو إعادة — وهي **ديناميكية**.

- [ ] افتراضياً **يستأذنك** قبل أي تعديل أو أمر — و`Shift+Tab` يبدّل الأوضاع.

- [ ] **Plan Mode** = **قراءة فقط** + خطة. راجع الخطة قبل كتابة الكود.

- [ ] قد يخطئ أو يعقّد الحل — **راجع المخرجات دائماً**. تخطي الصلاحيات له تكلفة.

- [ ] السياق ≈ مليون توكن، و**Auto-compact** عند الامتلاء — اعمل `/compact` مبكراً للأمان.

- [ ] `/compact` = ملخص + نافذة جديدة. `/clear` = بداية نظيفة تماماً.
  `/resume` = رجوع.

- [ ] `/rewind` بدل «رجّعني» لتوفير التوكنز (الكود / المحادثة / الاثنين).

- [ ] `/init` **أول خطوة** في أي مشروع ← **CLAUDE.md** في الـ **root** = دستور المشروع.

- [ ] خطّط بـ **Opus/Fable** ثم نفّذ بـ **Sonnet** لتوفير التوكنز.
- [ ] أي مهمة متكررة ← **Subagent** بسياق مستقل.
- [ ] **Skill** = اسم + **وصف (المُشغّل)** + خطوات، ولا تُحمَّل إلا عند الحاجة.
- [ ] `/doctor` للفحص، `/statusline` للمراقبة، `/diff` قبل الـ **commit**، `/cost` للتكلفة.

14 أسئلة المراجعة

حاول الإجابة أولاً دون النظر للإجابات، ثم صحّح نفسك من القسم التالي.

1 عرّف Claude Code في جملة واحدة. إجابة قصيرة

2 أيّ معادلة تصف الـ AI Agent كما شرحتها المحاضرة؟ اختيار من متعدد

أ) نموذج لغوي فقط

ب) نموذج لغوي + أدوات + ذاكرة + تخطيط داخل حلقة متكررة

ج) محرر أكواد + ترمنال

د) شات بوت + بحث على الإنترنت

3 النموذج اللغوي (مثل Opus أو Sonnet) هو الذي ينفّذ الأدوات بنفسه. صح أم خطأ

4 ما الفرق الجوهري بين Claude.ai (الشات) وClaude Code؟ إجابة قصيرة

5 رتّب خطوات الـ Agent Loop. إجابة قصيرة

6 لا يمكنك التدخل في عمل Claude Code إلا بعد أن ينتهي من الحلقة. صح أم خطأ

7 في أي وضع يقرأ Claude Code الملفات فقط ويرجع لك بخطة دون أي تعديل أو تنفيذ أوامر؟ اختيار من متعدد

أ) Default

ب) Accept Edits

ج) Plan Mode

د) Auto Mode

8 ما الاختصار الذي يبدّل بين أوضاع الصلاحيات؟ وما ترتيب الأوضاع في الشرح؟ إجابة قصيرة

9 ماذا تعني عبارة «تخطي الصلاحيات له تكلفة» (Skipping permissions has a cost)؟ إجابة قصيرة

10 ما الذي يفعله Claude Code تلقائياً عندما تمتلئ نافذة السياق؟ اختيار من متعدد

أ) يتوقف عن العمل

ب) يعمل Auto-compact فيلخّص ويحذف ما يمكن

ج) يحذف المشروع

د) يغيّر النموذج

11 ما الفرق بين `/compact` و`/clear`؟ إجابة قصيرة

12 كيف ترجع لمحادثة سابقة بعد عمل `/clear` أو `/new`؟ إجابة قصيرة

13 أين يوضع ملف CLAUDE.md؟ اختيار من متعدد

أ) على سطح المكتب

ب) في جذر (root) المشروع

ج) داخل مجلد node_modules

د) في إعدادات المتصفح

14 لماذا نحتاج CLAUDE.md؟ وماذا نكتب فيه؟ إجابة قصيرة

15 ما وظيفة الأمر `/init`؟ ومتى نستخدمه؟ إجابة قصيرة

16 ما الفرق بين Project Memory وUser Memory في `/memory`؟

إجابة قصيرة

17 تريد أن تخطط لمشروع كبير بجودة عالية وتوفّر التوكنز في نفس الوقت. ما الاستراتيجية التي اقترحتها المحاضرة؟ سؤال تطبيقي

18 أيّ من التالي **ليس** من مستويات `/effort`؟ اختيار من متعدد

أ) low
ب) xhigh
ج) ultra
د) max

19 ما هو الـ Subagent؟ وكيف تنشئه الآن بعد إزالة المعالج (wizard) من `/agents`؟ إجابة قصيرة

20 ما فائدة `/rewind`؟ وما خياراته؟ إجابة قصيرة

21 أيّ أمر تستخدمه لترى التغييرات غير الملتزم بها قبل الـ commit أو الرفع على GitHub؟ اختيار من متعدد

أ) `/diff`
ب) `/doctor`
ج) `/cost`
د) `/btw`

22 ما الذي يفحصه `/doctor`؟ ولماذا يهم «من يخاف على فلوسه»؟

إجابة قصيرة

23 مِمّ يتكوّن ملف الـ Skill؟ وما أهم جزء فيه ولماذا؟ إجابة قصيرة

24 اذكر الطرق الثلاث لاستدعاء Skill. إجابة قصيرة

25 الـ Skills تُحمَّل كاملة في نافذة السياق منذ بداية الجلسة. صح أم خطأ

26 ما الفرق بين الـ Skills وملف CLAUDE.md؟ إجابة قصيرة

27 عند تثبيت Plugin من الـ Marketplace، أيّ النطاقات التالية متاح؟

اختيار من متعدد

أ) Project / Local / User

ب) Cloud / Server

ج) Global / Team

د) Read / Write

28 أنشأت مجلداً جديداً اسمه test على سطح المكتب وتريد تشغيل Claude Code داخله من الترمنال. اكتب الأوامر. سؤال تطبيقي

29 ما فائدة `/btw`؟ إجابة قصيرة

30 شهرة Claude Code تعني أنه يمكن الاعتماد على مخرجاته دون مراجعة. صح أم خطأ

15 الإجابات النموذجية

1 **أداة برمجة تعمل بمنطق الوكيل (Agentic)** تقرأ مشروعك وتكتبه وتعدّل ملفاته وتشغّل أوامر الترمنال، وتتصل بأدوات المطوّر لتنجز المهام بسرعة.

2 **(ب)** نموذج لغوي (**LLM**) + أدوات (**Tools**) + ذاكرة (**Memory**) + تخطيط (**Planning**) يعمل داخل حلقة (**Loop**).

3 **خطأ.** النموذج لا يعرف استخدام الأدوات بنفسه؛ هو يُرجع «طلب استدعاء أداة» (**Tool Call**)، و**Claude Code** (الوكيل) هو الذي ينفّذ الأداة فعلياً.

4 في الشات أنت تكلّم النموذج والعملية قائمة على **النسخ واللصق**. أما **Claude Code** فوكيل له **وصول مباشر** لملفاتك وللترمنال ويستخدم الأدوات وينفّذ بدلاً منك.

5 ١) تكتب البرومت ← ٢) يجمع السياق عبر النموذج ← ٣) النموذج يرد بنص أو **Tool Call** ← ٤) ينفّذ إجراء (تعديل ملف / تشغيل أمر) ← ٥) يتحقق (**Verification**) ← ٦) إن تحقق المطلوب ينتهي (**Done**) وإلا يعيد الحلقة.

6 **خطأ.** الحلقة **ديناميكية**؛ تستطيع أثناء عمله إضافة معلومات أو سياق أو تغيير اتجاهه، ويأخذها في الحسبان فوراً.

7 **(ج) Plan Mode** — صلاحية قراءة فقط (**Read-only**)، يخطط ويرجع بقائمة مهام (**To-do list**).

8 `Shift + Tab`. الترتيب في الشرح: **Accept Edits On** ← **Plan Mode** ← **Auto Mode On** ← **Manual** (كل شيء بموافقتك).

9 كلما أعطيته حرية كاملة في تنفيذ الأوامر دون رجوع إليك، زاد احتمال حدوث **أخطاء يصعب اكتشافها**.

10 **(ب)** يعمل **Compact** تلقائياً: يقرر ما يلخّصه أو يحذفه ليرجع السياق لحجم مناسب ويتجنب الهلوسة وفقدان السياق.

11 `/compact` يأخذ **ملخص** المحادثة السابقة ويفتح نافذة سياق جديدة به. `/clear` يبدأ جلسة جديدة **بسياق فارغ تماماً** لا علاقة لها بما قبلها.

12 بالأمر `/resume` ثم تختار المحادثة من القائمة (وتوجد خانة بحث إن كثرت المحادثات).

13 **(ب)** في جذر المشروع، وهو ملف **Markdown**.

14 لأن كل جلسة تبدأ من الصفر فينسى القواعد ويعيد استكشاف الكود ويفترض افتراضات. الملف = **ذاكرة دائمة / دستور المشروع**: طريقة تشغيل المشروع، أسلوب الكود (مثل مسافتين للإزاحة، طريقة تسمية المتغيرات)، والتفضيلات الثابتة.

15 ينشئ ملف **CLAUDE.md** ويكتب فيه توثيق الكود الحالي. يُفضّل أن يكون **أول خطوة** مع **Claude** في أي مشروع.

16 **Project Memory** = ملف **CLAUDE.md** الخاص بالمشروع الحالي. **User Memory** = ذاكرة على **مستوى الجهاز/المستخدم** تنطبق على كل المشاريع. ويمكن تعديل أيٍّ منهما دون إغلاق الجلسة.

17 استخدم **نموذجاً قوياً** (**Opus** أو **Fable**) للتخطيط ووضع الخطة، ثم بدّل بـ `/model` إلى **نموذج أخف** (**Sonnet**) لتنفيذ الخطة؛ لأن النماذج الأعلى تستهلك توكنز أكثر.

18 **(ج) ultra**. المستويات: **low · medium · high · xhigh · max** وتعبّر عن عمق تفكير النموذج.

19 وكيل فرعي يعمل **بالتوازي بنافذة سياق خاصة به** لينجز مهمة محددة. تنشئه بطلب مباشر، مثل: «اعمل **subagent** اسمه **code-reviewer** يراجع الكود ويعطيني ملخص»، ثم تستدعيه بالكلمة المتفق عليها (مثل **review the code**).

20 يرجعك لنقطة محفوظة سابقاً إذا مشى **Claude** في اتجاه خاطئ، بدلاً من حرق توكنز بطلب «رجّعني». الخيارات: الكود فقط، المحادثة فقط، أو الاثنين معاً.

21 **(أ)** `/diff` — يعرض الـ **Uncommitted changes**.

22 **فحص صحة شامل** للإعدادات: هل الإعداد سليم؟ هل توجد **Skills** أو **MCP servers** محمّلة لا تستخدمها وتستهلك من السياق (أي من التوكنز)؟

23 الاسم (**name**) + الوصف (**description**) + الخطوات. **الوصف هو الأهم** لأنه المُشغِّل (**trigger**)؛ كلماته تحدد متى تُستدعى المهارة تلقائياً.

24 ١) ضمنياً: تذكر ما تريده فيستدعيها بالفهم الدلالي. ٢) بذكر اسمها في البرومت («استخدم **frontend-design skill**»). ٣) إجبارياً بالأمر `/` + اسمها.

25 **خطأ.** لا تُحمّل افتراضياً؛ تبقى «على جنب» ولا تُستدعى إلا عند الحاجة، فتبقى نافذة السياق فارغة.

26 **Skills** = طريقة لتنفيذ **مهمة محددة** وتُستدعى عند الحاجة فقط.
CLAUDE.md = **قواعد وفلسفة العمل** في المشروع كله، ويكون دائماً داخل السياق.

27 **(أ) Project scope** (المشروع فقط) أو **Local scope** أو **User scope** (المستخدم على الجهاز كله). وبعد التثبيت نفّذ `/reload-plugins`.

28 `cd Desktop` ← `mkdir test` ← `cd test` ← `claude`. (و`ls` لعرض محتويات المجلد الحالي، و`exit` للخروج من **Claude**).

29 تسأل سؤالاً جانبياً **دون أن تقطع** المحادثة أو المهمة الجارية.

30 **خطأ.** قد يفهم قصدك خطأً أو يُحدث **bug** أو يعقّد الحل (**over-engineering**). أي أداة تخطئ؛ **راجِع المخرجات دائماً.**

16 عناوين للتوسع والبحث

موضوعات مرتبطة بالمحاضرة تستحق التعمق، ومعها عبارة بحث مقترحة:

هندسة السياق (Context Engineering)

كيف تدير نافذة السياق بذكاء: متى تعمل **compact**، تقسيم المهام، وتقليل الضوضاء لتحسين جودة الإجابات.

`Claude Code context management best practices`

كتابة ملف CLAUDE.md احترافي

أمثلة لملفات حقيقية: أوامر البناء والاختبار، أسلوب الكود، القواعد الممنوعة، وتنظيم الذاكرة بين المشروع والمستخدم.

`CLAUDE.md examples`

بناء Skills خاصة بك

كتابة ملف **SKILL.md**، صياغة وصف يُشغّل المهارة في الوقت الصحيح، واستخدام **skill-creator**.

`Claude Agent Skills SKILL.md`

الوكلاء الفرعيون (Subagents)

تصميم وكلاء متخصصين (مراجعة كود، اختبارات، توثيق) وتشغيلهم بالتوازي وتوزيع الأدوات عليهم.

`Claude Code subagents`

خوادم MCP (Model Context Protocol)

ربط **Claude Code** بأدوات خارجية (قواعد بيانات، **GitHub**، متصفح) ومتى تستهلك من السياق.

`Model Context Protocol servers`

الـ Hooks والأتمتة

تشغيل أوامر تلقائياً قبل/بعد استخدام الأدوات (مثل تشغيل الـ **linter** بعد كل تعديل).

`Claude Code hooks`

أوضاع الصلاحيات والأمان

ضبط قوائم السماح والمنع في **settings.json**، ومخاطر تخطي الصلاحيات، والعمل في بيئات معزولة.

`Claude Code permissions settings.json`

أساسيات الترمنال وGit

أوامر الملفات والمجلدات، ثم **commit / branch / push / diff** لأن **Claude Code** يعمل معها باستمرار.

`Git basics for beginners`

اختيار النموذج ومستوى الجهد

متى تستخدم **Opus** أو **Sonnet** أو **Haiku**، وأثر **effort** على الجودة والتكلفة.

`Claude models comparison`

Ultraplang Plan Mode

سير عمل «خطّط ثم نفّذ»، ومناقشة الخطة على الويب قبل كتابة سطر كود.

`Claude Code plan mode workflow`

Claude Code على الويب وGitHub

تشغيل جلسات سحابية، ربط المستودعات، وفتح **Pull Requests** تلقائياً.

`Claude Code on the web GitHub`

شهادة كورس Anthropic

المحاضرة مبنية على كورس **Claude Code 101**؛ أكمل الكورس الرسمي وحل الأسئلة للحصول على الشهادة.

`Anthropic Academy Claude Code 101`

تم إعداد هذا الملخص من محتوى المحاضرة · **Claude Code** للمبتدئين — غريب الشيخ